

### 3) 叠加组装

叠加组装发生在两个或两个以上构件放在一起创建一个新构件的时候。这个新构件合并了原构件的功能，从而对外提供了新的接口。外部应用可以通过新接口来调用原有构件的接口，而原有构件不互相依赖，也不互相调用。这种组装方式适合于构件是程序单元或者构件是服务的情况。

当创建一个系统时，可能会用到所有的构件组装方式，对所有情况都必须编写胶水代码来连接构件。而当编写构件时，尤其是为了组装来编写构件时，可能经常会面临接口不兼容的问题，即所要组装的构件的接口不一致。一般会出现3种不兼容情况：

- (1) 参数不兼容。接口每一侧的操作有相同的名字，但参数类型或参数个数不相同。
- (2) 操作不兼容。提供接口和请求接口的操作名不同。
- (3) 操作不完备。一个构件的提供接口是另一个构件请求接口的一个子集，或者相反。

针对上述不兼容情况，必须通过编写适配器构件来解决不兼容的问题，适配器构件使两个可复用构件的接口相一致；适配器构件将一个接口转换为另外一个接口。

当用户选择组装方式时，必须考虑系统所需要的功能性需求、非功能性需求，以及当系统发生改变时，一个构件能被另一个构件替代的难易程度。

通过购买商用构件（平台）并遵循其开发标准来进行应用开发，是提高应用软件开发效率的常见选择。

## 4.6.2 中间件

中间件是一种独立的系统软件或服务程序，分布式应用软件借助这种软件在不同的技术之间共享资源，中间件位于操作系统之上，管理计算资源和网络通信，实现应用之间的互操作。具体来说，中间件的基本功能包括以下6个方面：

- (1) 负责客户机与服务器之间的连接和通信，以及客户机与应用层之间的高效率通信机制。
- (2) 提供应用层不同服务之间的互操作机制，以及应用层与数据库之间的连接和控制机制。
- (3) 提供一个多层架构的应用开发和运行的平台，以及一个应用开发框架，支持模块化的应用开发。
- (4) 屏蔽硬件、操作系统、网络和数据库的差异。
- (5) 提供应用的负载均衡和高可用性、安全机制与管理功能，以及交易管理机制，保证交易的一致性。
- (6) 提供一组通用的服务去执行不同的功能，避免重复的工作并使应用之间可以协作。

### 1. 中间件的分类

在不同的角度或不同的层次上，对中间件的分类会有所不同。

采用自底向上的方式来划分，可分为底层中间件、通用型中间件和集成型中间件3个层次。底层中间件的主流技术主要有Java虚拟机（Java Virtual Machine, JVM）、公共语言运行库（Common Language Runtime, CLR）、自适应通信环境（Adaptive Communication Environment, ACE）等；通用型中间件也称为平台，其主流技术主要有RPC、ORB、面向消息的中间件



(Message-Oriented Middleware, MOM) 等;集成型中间件的主流技术主要有 Workflow、EAI 等。

按照中间件在分布式系统中承担的职责不同,可以将中间件划分成以下几类。

(1) 通信处理(消息)中间件。正如安装红绿灯、设立交通管理机构、制定交通规则才能保证道路交通畅通一样,在分布式系统中,人们要建网和制定通信协议,以保证系统能在不同平台之间通信,实现分布式系统中可靠的、高效的、实时的跨平台数据传输,这类中间件称为消息中间件,它也是市面上销售额最大的中间件产品,目前主要产品有 BEA 的 eLink、IBM 的 MQSeries、TongLINK 等。实际上,一般的网络操作系统(如 Windows)已包含了其部分功能。

(2) 事务处理(交易)中间件。在分布式事务处理系统中,经常要处理大量事务,特别是 OLTP 中,每项事务常常要多台服务器上的程序按顺序协调完成,一旦中间发生某种故障,不但要完成恢复工作,而且要自动切换系统,达到系统永不停机,实现高可靠性运行。要使大量事务在多台应用服务器上能实时并发运行,并进行负载平衡的调度,为了实现这个目标,要求中间件系统具有监视和调度整个系统的功能。

(3) 数据存取管理中间件。在分布式系统中,重要的数据都集中存放在数据服务器中,它们可以是关系型的、复合文档型、具有各种存放格式的多媒体型的,或者是经过加密或压缩存放的,该中间件将为在网络上虚拟缓冲存取、格式转换、解压等带来方便。

(4) Web 服务器中间件。浏览器图形用户界面已成为公认规范,然而它的会话能力差、不擅长做数据写入、受 HTTP 协议的限制等,必须进行修改和扩充,就形成了 Web 服务器中间件,如 Silverstream 公司的产品。

(5) 安全中间件。一些军事部门、政府部门和商务部门上网的最大障碍是安全保密问题,而且不能使用国外提供的安全措施(如防火墙、加密、认证等),必须用国产产品。产生不安全因素是由操作系统引起的,但必须要用中间件去解决,以适应灵活多变的要求。

(6) 跨平台和架构的中间件。当前开发大型应用软件通常采用基于架构和构件的技术,在分布式系统中,还需要集成各节点上的不同系统平台上的构件或新老版本的构件,由此产生了架构中间件。比如, CORBA 的功能强大,可以跨任意平台,但是过于庞大; JavaBeans 较灵活简单,很适合于做浏览器,但运行效率有待改善; COM+ 模型主要适合 Windows 平台,在桌面系统已广泛使用。由于国内新建系统多基于 UNIX(包括 Linux)和 Windows,因此,针对这两个平台建立了相应的中间件市场。

(7) 专用平台中间件。为特定应用领域设计领域参考模式,建立相应架构,配置相应的构件库和中间件,为应用服务器开发和运行特定领域的关键任务(如电子商务、网站等)。

(8) 网络中间件。它包括网管、接入、网络测试、虚拟社区、虚拟缓冲等,也是当前最热门的研发项目。

## 2. 中间件的应用

中间件提供了应用系统基本的运行环境,也为应用系统提供了更多的高级服务功能。例如,名字服务、事件服务、通告服务、日志服务等。在这些服务之上,还需要考虑不同行业的需求、不同的应用领域。



中间件技术在企业应用集成中扮演着重要的角色，可以从不同层次采用不同种类、不同技术的中间件产品进行应用集成。

### 3. 中间件与构件的关系

从本质上来说，中间件是对分布式应用的抽象，它抛开了与应用相关的业务逻辑细节，保留了典型的分布交互模式的关键特征。经过抽象，将纷繁复杂的分布式系统进行提炼和必要的隔离后，以统一的形式呈现给应用。应用在中间件提供的环境中可以更好地集中于业务逻辑，并以构件的形式存在，最终自然而然地在异构环境中实现良好的协同工作。

中间件遵循特定的构件标准，为构件互连提供支持，并提供相应的公共服务。中间件支持的连接子实现有如下优势：

(1) 中间件提供了构件之间跨平台交互的能力，且遵循特定的工业标准，如 CORBA、J2EE、COM 等，可以有效地保证构件之间的通信完整性。

(2) 产品化的中间件可以提供强大的公共服务能力，这样能够更好地保证最终系统的质量属性。随着中间件技术的发展，也产生了一类新的架构风格，即中间件导向的体系结构风格 (Middleware-Induced Architectural Style)。

中间件与软件架构实际上是从两种不同的角度看待软件的中间层次。从某种程度上说，中间件就是架构，是构件存在的基础，中间件促进了构件化的实现。从以下几个方面解释中间件与构件的关系：

(1) 面向需求。基于架构的构件化软件开发应当是面向需求的，即设计师将精力集中于业务逻辑本身，而不必为分布式应用中的非功能质量属性耗费大量的精力，理想的架构在这些方面应当为软件提供良好的运行环境。事实上，这些正是中间件所要解决的问题，因此，基于中间件开发的应用是真正面向需求的，从本质上符合构件化设计的思想。

(2) 业务的分隔和包容性。服务器构件要求有很好的业务自包容性，应用开发人员可以按照不同的业务进行功能的划分，体现为不同的接口或交互模式。针对每种业务，设计和开发是可以独立进行的。在提供业务的分隔和包容性方面，架构和中间件有同样的目标。例如，消息中间件规定了消息是有属性的，其中部分属性则与业务的划分有关。构件只进行相应类型的消息交互，至于如何保证业务的分类运行与管理，则是中间件的任务。

(3) 设计与实现分离。构件对外发生作用或构件间的交互，都是通过接口进行的，构件使用者只需要知道构件的接口，而不必关心其内部实现，这是设计与实现分离的关键。中间件在分布交互模式上也规定了接口（或类似）机制，例如，IDL (Interface Definition Language, 接口定义语言) 就是描述接口的语言规范，从早期的分布式计算环境 (Distributed Computing Environment, DCE) 到现在的 CORBA、DCOM、RMI 等都使用 IDL 描述接口，所不同的只是语言规范。

(4) 隔离复杂的系统资源。架构很重要的一个功能就是将系统资源与应用构件隔离，这是保证构件可复用甚至即插即用的基础，与中间件的意图也是一致的。中间件最大的优势之一就是屏蔽多样的系统资源，保证良好的互操作性。应用构件开发人员只需要按照中间件规定的模式进行设计开发，而不必考虑下层的系统平台。因此，中间件真正提供了与环境隔离的构件开发模式。

(5) 符合标准的交互模型。架构是一种抽象的模型，但模型中应当定义一些可操作的成分。



例如，标准的协议等。中间件则实现了架构的模型，实现了标准的协议。例如，基于 CORBA 的中间件使用的是 CORBA 规范作为架构模型，定义了数据表示语法、数据包格式、消息语义等内容，以实现互操作性协议。因此，基于中间件的构件是符合标准模型的。

(6) 软件复用。软件复用是构件化软件开发的根本目标之一，中间件提供了构件封装、交互规则、与环境的隔离等机制，这些都为软件复用提供了方便的解决方案。另外，中间件可以建立访问过去的应用的通道，或者在新的中间件体系中建立特殊的运行容器，封装以往的应用，从而最终做到对遗留系统的继承性复用。

(7) 提供对应用构件的管理。基于中间件的软件可以方便地进行管理，因为构件总可以通过标识机制进行划分。例如，COM 就是利用 Windows 系统注册表配合几种唯一标识构件的方式，实现构件的登记、注销和定位。CORBA 规范中有接口池、实现池等规范定义，配合应用登记管理的机制，也能对应用构件实施管理。

总之，基于中间件开发的应用是构件化的，中间件提供了构件的架构，提高了应用构件开发的效率和质量。中间件作为应用软件系统集成的关键技术，保证了构件化思想的实施，并为构件提供了真正的运行空间。反之，构件对新一代中间件产品也起到了促进作用，构件化的中间件在市场上具有强大的生命力。

## 4.7 Web 服务

所谓 Web 服务是指由企业发布的、完成其特别商务需求的在线应用服务，其他公司或应用软件能够通过互联网来访问并使用这种在线服务。

关于 Web 服务的定义有很多种，其核心是面向服务的架构（Service Oriented Architecture, SOA），即通过完善的接口实现远程访问应用。通过提供简单和普遍适用的标准，Web 服务可以作为信息系统集成及系统间自动化信息交换的基础。

从技术角度来讲，Web 服务是一种可以用来解决跨越网络的应用集成问题的开发模式，这种模式为实现“软件即服务”提供了技术保障。

### 1. Web 服务的体系结构

Web 服务的体系结构如图 4-13 所示，其中，服务提供者（服务器）和服务请求者（客户端）是必需的，服务注册中心是可选的角色。

服务提供者是 Web 服务的所有者，如企业、ICP（Internet Content Provider，互联网内容提供商）等，该角色负责定义并实现 Web 服务，使用服务描述语言对 Web 服务进行详细、准确、规范的描述，并将该描述发布到服务注册中心，供服务请求者查找并绑定使用。

服务请求者是 Web 服务的使用者。虽然 Web 服务面向的是程序，但程序的最终使用者仍然

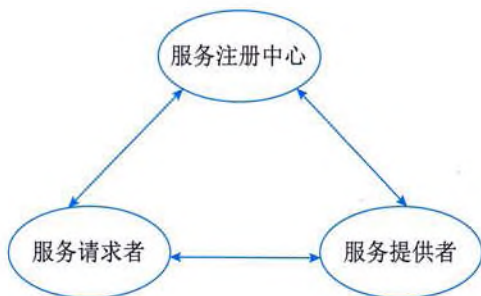


图 4-13 Web 服务的体系结构



是企业或用户，因此，服务请求者是查找、绑定并调用服务或与服务进行交互的应用程序，该角色可由浏览器担当，由人或程序来控制。

服务注册中心是连接服务提供者和服务请求者的纽带，在某些情况下，该角色是可选的。例如，使用静态绑定的 Web 服务，服务提供者可直接把描述发送给服务请求者。若没有服务注册中心，服务请求者则可以从其他来源得到服务描述，例如文件、FTP 站点、Web 站点等。

Web 服务模型中的操作包括以下 3 种：发布服务描述、查找服务描述、根据服务描述绑定或调用服务。

## 2. Web 服务的技术平台

Web 服务平台由支持 Web 服务的基本技术和协议组成，主要包括 HTTP、XML、SOAP、WSDL、UDDI、WSIF 和 WSFL 等。它们构成了发现 and 调用一个独立的 Web 服务所提供功能的标准机制的基础。Web 服务栈以层次结构来表示，高层在低层的基础之上构建，如图 4-14 所示。

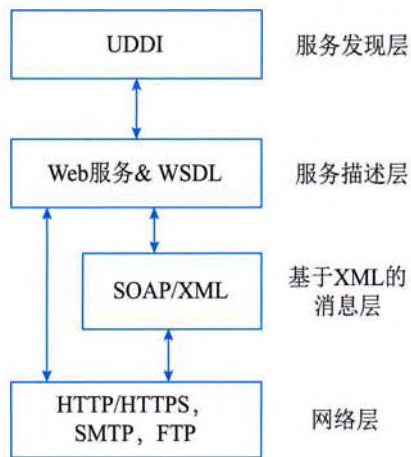


图 4-14 Web 服务栈

### 1) HTTP

超文本传输协议 (HTTP) 是互联网上应用最为广泛的一种网络协议。所有的 WWW 文件都必须遵守这个标准。

### 2) XML

XML (eXtensible Markup Language, 可扩展标记语言) 是一种数据存储语言，使用一系列简单的标记描述数据。XML 是互联网环境中跨平台的、依赖于内容的技术，是当前处理结构化文档信息的有力工具。

### 3) SOAP

SOAP 是目前被广泛接受的消息传输协议，用于在网络应用程序之间交换结构化数据，是一种基于 XML 的机制。SOAP 主要是在分布的、分散的环境中提供了一个跨互联网调用服务的框架结构，并提供了独立于编程语言和分布对象底层基础结构的跨平台集成机制。

### 4) WSDL

Web 服务的接口由基于 XML 的 WSDL 定义，它提供了应用访问指定的 Web 服务所必需的全部信息，描述服务提供了什么功能、服务位于何处以及服务如何调用。

WSDL 以 XML 格式描述网络服务，将服务描述为在包含面向过程或面向文档信息的消息上进行操作的一组端点。操作和消息是抽象描述的，然后绑定到具体的网络协议和消息格式以定义一个端点。相关的具体端点被组合成为抽象端点 (服务)。WSDL 是可扩展的，允许描述任何端点和消息，而不考虑通信使用的消息格式或网络协议。

Web 服务的目标之一是允许应用程序以标准的方式在两个或多个同等的服务之间进行选择，因为有时应用可以由作为支持网络的服务而实现的构件构造而成，甚至可以从这些服务中进行动态选择。服务描述层定义了为程序提供足够信息所需的描述机制，使程序能够根据一定的准则选择服务，如服务的质量、安全性、可靠性等。



### 5) UDDI

面对极为丰富的服务，最常出现的问题是“在哪里以及如何找到需要的信息”。UDDI 规范在底层协议的基础上又定义了一层，在这一层，不同的企业能够以相同的方式描述自己提供的服务和查询对方提供的服务。

UDDI 是一套基于 Web 的、分布式的、为 Web 服务提供的信息注册中心实现标准规范，同时也包含一组使企业能将自身提供的 Web 服务注册以使别的企业能够发现的访问协议的实现标准。

Web 服务不仅在信息系统间提供了一种新的通信方式，而且还带来了针对软件组件的全新思维方式。在一个系统上所使用的服务将演变成为可在其他系统上重复使用的组件。充分利用现有信息系统中的功能，并将它们转变成可以以全新组合方式重复使用的组件，能够显著降低新应用的构建成本。

## 4.8 云计算

云计算指通过计算机网络（多指因特网）形成的计算能力极强的系统，可存储、集合相关资源，并可按需配置，向用户提供个性化服务。从狭义上讲，云计算就是一种提供资源的网络，使用者可以随时获取“云”上的资源，按需求量使用，并且“云”可以看成是无限扩展的，只要按使用量付费。从广义上说，云计算是与信息技术、软件、互联网相关的一种服务，这种计算资源共享池叫做“云”，云计算把许多计算资源集合起来，通过软件实现自动化管理，只需要很少的人参与就能快速提供资源。

### 1. 云计算的服务方式

产业界和学术界目前一致认为云计算有 3 类典型的服务方式，自上而下分别为软件即服务（Software as a Service, SaaS）、平台即服务（Platform as a Service, PaaS）和基础设施即服务（Infrastructure as a Service, IaaS）。

#### 1) 软件即服务

在 SaaS 的服务模式下，服务提供商将应用软件统一部署在云计算平台上，客户根据需要通过互联网向服务提供商订购应用软件服务，服务提供商根据客户所订购软件的数量、时间的长短等因素收费，并且通过标准浏览器向客户提供应用服务。

#### 2) 平台即服务

在 PaaS 模式下，服务提供商将分布式开发环境与平台作为一种服务来提供。这是一种分布式平台服务，厂商提供开发环境、服务器平台、硬件资源等服务给客户，客户在服务提供商平台的基础上定制开发自己的应用程序，并通过其服务器和互联网传递给其他客户。

#### 3) 基础设施即服务

在 IaaS 模式下，服务提供商将多台服务器组成的“云端”基础设施作为计量服务提供给客户。具体来说，服务提供商将内存、I/O 设备、存储和计算能力等整合为一个虚拟的资源池，为客户提供所需要的存储资源、虚拟化服务器等服务。

对这 3 种服务方式进行分析后，可以看出这 3 种服务模式有如下特征：



(1) 在灵活性方面, SaaS → PaaS → IaaS 灵活性依次增强。这是因为用户可以控制的资源越来越底层, 粒度越来越小, 控制力增强, 灵活性也增强。

(2) 在方便性方面, IaaS → PaaS → SaaS 方便性依次增强。这是因为 IaaS 只是提供 CPU、存储等底层基本计算能力, 用户必须在此基础上针对自身需求构建应用系统, 工作量较大, 方便性较差。而 SaaS 模式下, 服务提供商直接将具有基本功能的应用软件提供给用户, 用户只要根据自身应用的特定需求进行简单配置后就可以使应用系统上线, 工作量较小, 方便性较好。

(3) PaaS 是云计算服务模式中最为关键的一层, 在整个云计算体系中起着支撑的作用。PaaS 通常以特定的互联网资源为中心, 采用开放平台的形式, 对外提供基于 Web 的 API 服务。PaaS 的地位相当于系统软件, 需要为上层 SaaS 应用提供 API, 以支持各种 SaaS 应用的开发。除了一些基础性的 API 之外, PaaS 还要提供更多高级的服务型 API。这样, 上层的应用就可以利用这些高级服务, 构建面向最终用户的具体应用。

## 2. 云计算的部署模式

根据美国国家标准与技术研究院 (National Institute of Standards and Technology, NIST) 的定义, 云计算从部署模式上看可以分为公有云、社区云、私有云和混合云 4 种类型。

### 1) 公有云

在公有云模式下, 云基础设施是公开的, 可以自由地分配给公众。企业、学术界与政府机构都可以拥有和管理公有云, 并实现对公有云的操作。

公有云能够以低廉的价格为最终用户提供有吸引力的服务, 创造新的业务价值。作为支撑平台, 公有云还能够整合上游服务 (如增值业务, 广告) 提供商和下游终端用户, 打造新的价值链和生态系统。

### 2) 社区云

在社区云模式下, 云基础设施分配给一些社区组织所专有, 这些组织共同关注任务、安全需求、政策等信息。云基础设施被社区内的一个或多个组织所拥有、管理及操作。

社区云是公有云范畴内的一个组成部分, 指在一定的地域范围内, 由云计算服务提供商统一提供计算资源、网络资源、软件和服务能力所形成的云计算形式。即基于社区内的网络互连优势和技术易于整合等特点, 通过对区域内各种计算能力进行统一服务形式的整合, 结合社区内的用户需求共性, 实现面向区域用户需求的云计算服务模式。

### 3) 私有云

在私有云模式下, 云基础服务设施分配给由多种用户组成的单个组织。它可以被这个组织或其他第三方组织所拥有、管理及操作。

### 4) 混合云

混合云是公有云、私有云和社区云的组合。出于安全和控制原因, 并非所有的企业信息都能放置在公有云上, 因此企业将会使用混合云模式, 将公有信息和私有信息分别放置在公有云和私有云环境中。在混合云构建方面, 大部分企业选择同时使用公有云和私有云, 也有一些会同时建立社区云。

## 第5章 数据库系统

在当今的知识经济时代，信息是经济发展的战略资源，信息技术已经成为社会生产力中重要的组成部分。人们充分认识到，数据库是信息化社会中信息资源管理与开发利用的基础，当今的计算机信息系统也都以数据库技术为基础。对于一个国家来说，数据库的建设规模和使用水平已成为衡量该国信息化程度的重要标志。因此，数据库课程是计算机领域中的一门重要课程，也是系统分析师必须要掌握的专业知识与技能。

### 5.1 数据库管理系统

数据库是长期存储在计算机内的、有组织的、可共享的数据集合，数据库系统是指在计算机信息系统中引入数据库后的系统，一般由数据库、数据库管理系统（DataBase Management System, DBMS）、应用系统、数据库管理员（DataBase Administrator, DBA）和用户构成。

#### 5.1.1 概述

数据库管理系统是一种操纵和管理数据库的大型软件，用于建立、使用和维护数据库。它对数据库进行统一的管理和控制，以保证数据库的安全性和完整性。用户通过 DBMS 访问数据库中的数据，数据库管理员也通过 DBMS 进行数据库的维护工作。DBMS 可以支持多个应用程序和用户用不同的方法在同时或不同时刻去建立、修改和询问数据库。大部分 DBMS 提供数据定义语言（Data Definition Language, DDL）和数据操作语言（Data Manipulation Language, DML），供用户定义数据库的模式结构与权限约束，实现对数据的追加、删除等操作。

简单地说，数据库管理系统是一个能够提供数据录入、修改、查询的数据操作软件，具有数据定义、数据操作、数据存储与管理、数据维护、通信等功能，且能够允许许多用户使用。

#### 5.1.2 三级划分法

数据库系统的结构划分可以有多种不同的层次或不同的角度，典型的是三级划分法，就是把数据库系统划分为三级模式，在三级模式之间提供两级映射，这两级映射保证了数据库中的数据具有较高的物理独立性和逻辑独立性。

##### 1. 三级模式

数据库系统的三级模式如图 5-1 所示，从图 5-1 中可以看出，数据库系统由外模式、概念模式和内模式三级构成。



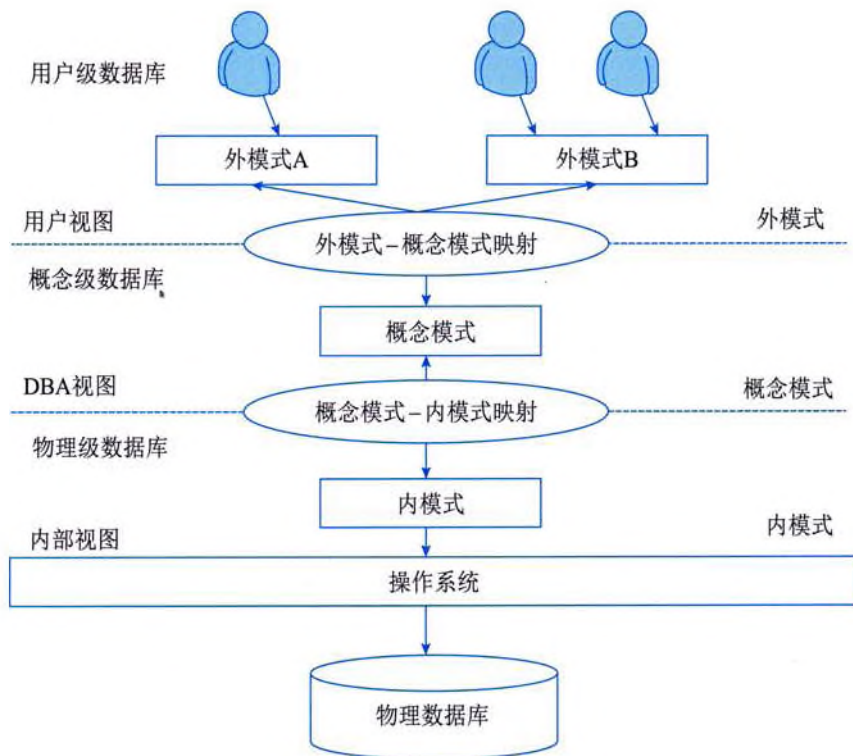


图 5-1 数据库系统结构层次图

外模式也称为子模式或用户模式，对应于用户级数据库。外模式用以描述用户（包括程序员和终端用户）看到或使用的那部分数据的逻辑结构，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。用户根据外模式用数据操作语句或应用程序去操作数据库中的数据。外模式主要描述组成用户视图的各个记录的组成、相互关系、数据项的特征、数据的安全性和完整性约束条件。一个数据库可以有多个外模式，一个应用程序只能使用一个外模式。

概念模式也称为模式或逻辑模式，对应于概念级数据库。概念模式是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图，用以描述现实世界中的实体及其性质与联系，定义记录、数据项、数据的完整性约束条件及记录之间的联系。概念模式通常还包含访问控制、保密定义和完整性检查等方面的内容，以及概念/物理之间的映射。一个数据库只有一个概念模式。

内模式对应于物理级数据库，是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式。内模式不同于物理层，它假设外存是一个无限的线性地址空间。内模式定义的是存储记录的类型、存储域的表示和存储记录的物理顺序，以及索引和存储路径等数据的存储组织。一个数据库只有一个内模式。

在数据库系统的三级模式中，模式是数据库的中心与关键；内模式依赖于模式，独立于外模式和存储设备；外模式面向具体的应用，独立于内模式和存储设备；应用程序依赖于外模式，